

Programación Lineal II

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

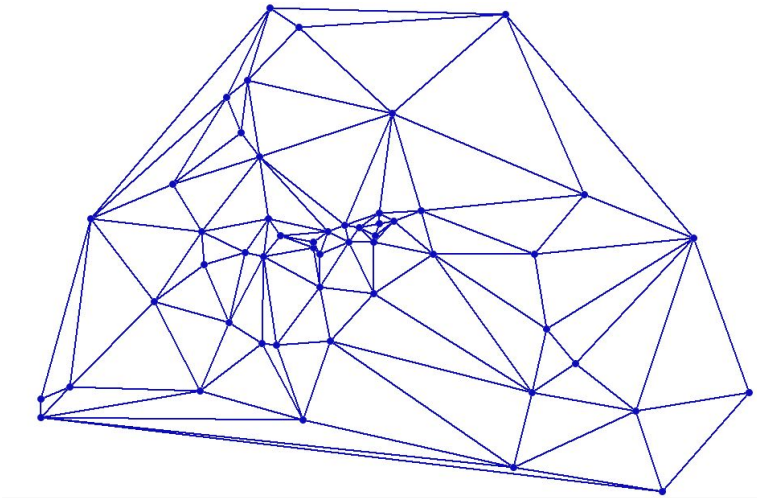
Repaso de la clase anterior



Programación Lineal

- *Es una técnica para resolver problemas de optimización.*
- *Genera un modelo de inecuaciones lineales a cumplir y una función objetivo a maximizar ó minimizar.*
- *Todas las restricciones deben ser LINEALES.*
- *Las variables pueden ser continuas, enteras o bivalentes.*
- *Una vez planteado el modelo, se puede resolver:*
  - *Gráficamente (si tengo sólo dos variables)*
  - *Por software*
  - *Algorítmicamente (lo vamos a ver hoy)*

Resolución de Programación Lineal Entera: Problemas combinatorios



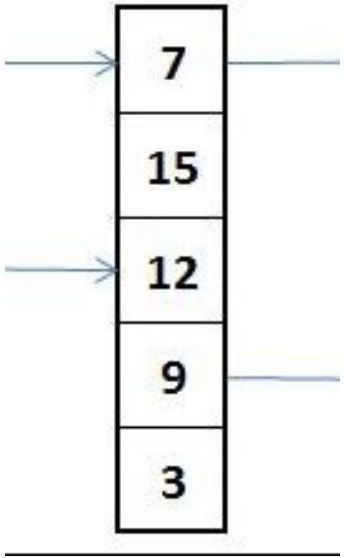
Ejemplo de Programación Lineal

Tenemos un array A de números enteros (positivos, negativos o cero) no ordenado. Queremos encontrar la secuencia de uno o más elementos contiguos que sumen el máximo valor posible.



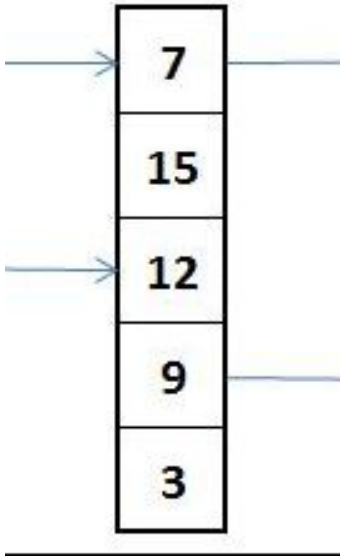
Ejemplo de Programación Lineal

- *¿Cuáles son las variables?*
- *¿Cuál es el objetivo?*
- *¿Cuáles son las restricciones?*



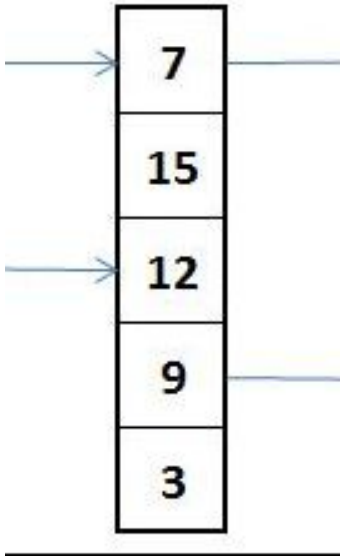
Ejemplo de Programación Lineal

- *Determinar qué elementos seleccionar.*
- *Maximizando la suma de sus valores.*
- *Cumpliendo con que sean elementos consecutivos.*
- *¿Cómo?*
- *¿CÓMO?*



Ejemplo de Programación Lineal

- *Sean Yi e Yj, no consecutivos, pero i < j*
- *Sólo puedo incluir a ambos si también incluyo a todos los Yk intermedios.*



Ejemplo de Programación Lineal

- Y_i : *Vale 1 si elijo el elemento i.*
- $Z(max) = \sum Ci.Y_i$ *(para i=1..n)*

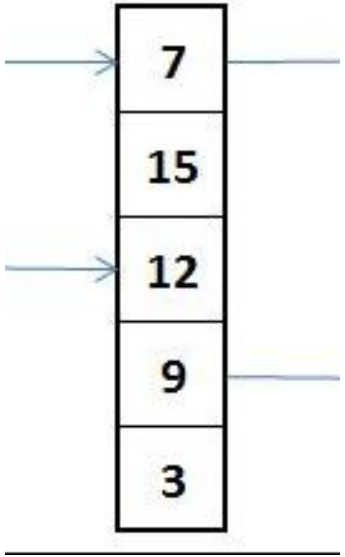
- *2 YANDij <= Yi + Yj <= 1 + YANDij*
- *YANDij <= Yk*
  - *para todo $i < k < j$*



PROBLEMA DE SELECCIÓN

Ejemplo de Programación Lineal

- *Para todo Yk:*
  - *Si Yk = 0, puedo elegir elementos anteriores o posteriores, pero no ambos*
  - *Si Yk = 1, puedo elegir los que quiera*



Ejemplo de Programación Lineal

- Y_i : *Vale 1 si elijo el elemento i.*
- $Z(max) = \sum Ci.Y_i$ *(para i=1..n)*



- *YAntk <= $\sum Y_i$ (para i=1..k-1) <= (k-1) YAntk*
- *YDespk <= $\sum Y_j$ (para j=k+1..n) <= (n-k+1) YDespk*
- *YAntk + Y Despk <= 1*
  - *para todo k: 2.....n-1*

Ejemplo de Programación Lineal ¿Cuál preferimos?

- Y_i : *Vale 1 si elijo el elemento i.*
- $Z(max) = \sum Ci.Y_i$ *(para i=1..n)*

- *2 YANDij <= Yi + Yj <= 1 + YANDij*
- *YANDij <= Yk*
  - *para todo i < k < j*

- *YAntk <=* $\sum Y_i$ *(para i=1..k-1) <= (k-1) YAntk*
- *YDespk <=* $\sum Y_j$ *(para j=k+1..n) <= (n-k+1) YDespk*
- *YAntk + Y Despk <= 1*
  - *para todo k: 2.....n-1*

Resolución de Programación Lineal Entera: Problema del Viajante de Comercio



Programación Lineal Entera: Viajante

- *Determinar en qué orden visitar las ciudades.*
- *Para minimizar el costo del viaje.*
- *Visitando cada una exactamente una vez y volviendo al inicio.*

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Programación Lineal Entera: Viajante

- Y_{ij}: *Vale 1 si hago el trayecto de i a j* $\forall\ i,j: 0...N$

  - $Z(Min) = \sum_i \sum_j C_{ij} \cdot Y_{ij}$ $\forall\ i,j: 0...N$
    (C_{ij} *es constante*)
  - $\sum_j Y_{ij} = 1$ $\forall\ i: 0...N$
  - $\sum_i Y_{ij} = 1$ $\forall\ j: 0...N$

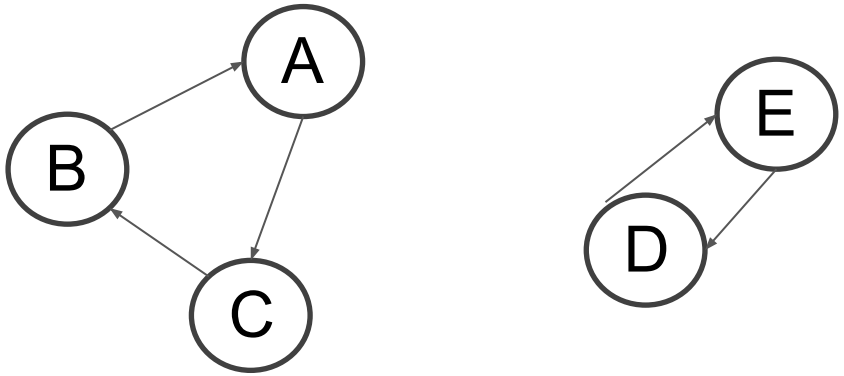
	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

N es la cantidad de ciudades a visitar, además del origen. En este ejemplo, es 4

No es suficiente

Programación Lineal Entera: Viajante

- *Una solución podría ser:*

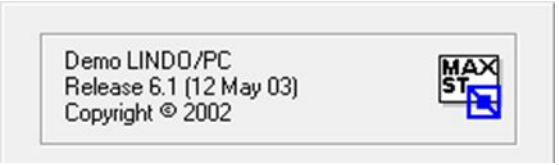


	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Programación Lineal Entera: Viajante

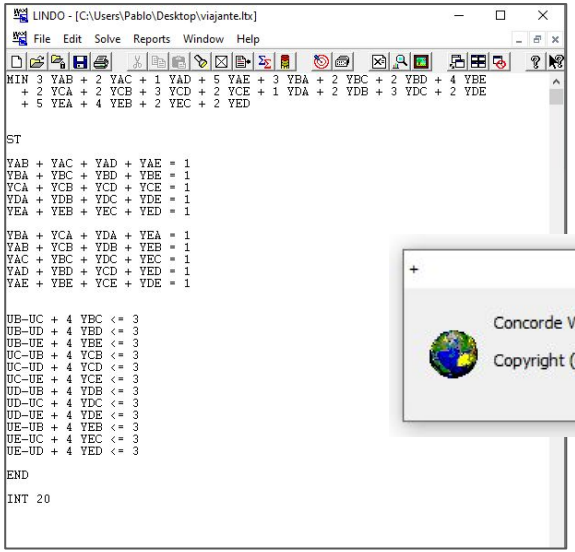
- *Para evitarlo, agregamos las restricciones MTZ:*
  - *Variables U_i: Indican el orden en que visito cada ciudad (menos la inicial)*
  - $U_i - U_j + N\,Y_{ij} <= N\text{-}1$ ($\forall\ i, j$ *salvo el origen)*

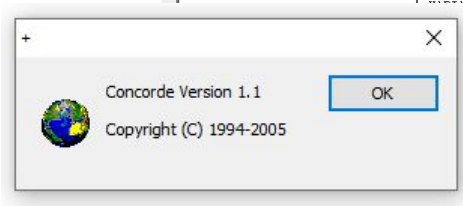
	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

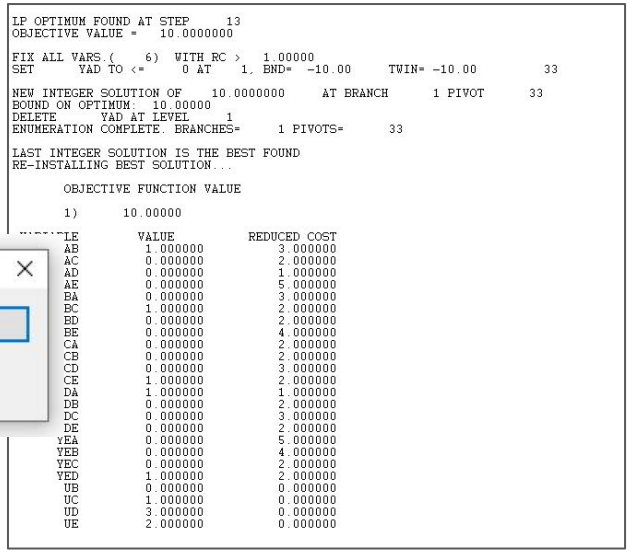


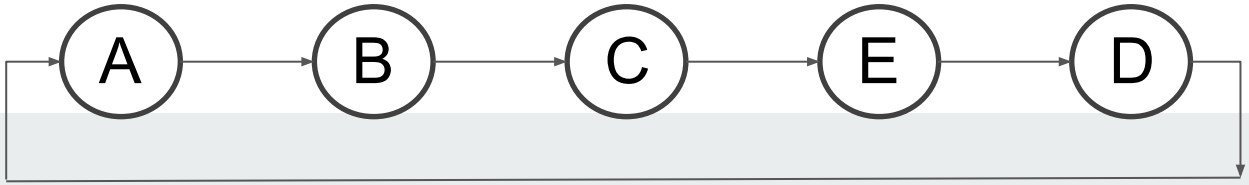
Programación Lineal Entera: Viajante

- *Resolución con LINDO*









Programación Lineal Entera: Viajante

- *Resolución con la biblioteca PULP de Python*

Shell

```
Result - Optimal solution found

Objective value:                9913.00000000
Enumerated nodes:               34493
Total iterations:               936799
Time (CPU seconds):             15.15
Time (Wallclock seconds):       15.15

Option for printingOptions changed from normal to all
Total time (CPU seconds):       15.16   (Wallclock seconds):       15.16

Y_Buenos_Aires_Corrientes 917
Y_La_Plata_Buenos_Aires 58
Y_Santa_Rosa_Neuquén 529
Y_Santa_Fe_Córdoba 369
Y_Paraná_Santa_Fe 31
Y_Córdoba_La_Rioja 450
Y_San_Luis_Santa_Rosa 505
Y_San_Juan_Mendoza 170
Y_Mendoza_San_Luis 258
Y_La_Rioja_San_Juan 444
Y_Viedma_La_Plata 904
Y_Neuquén_Rawson 743
Y_Rawson_Ushuaia 1735
Y_Rio_Gallegos_Viedma 1642
Y_Ushuaia_Rio_Gallegos 579
Y_Corrientes_Paraná 579
U_La_Plata 15.0
U_La_Plata 15.0
U_Santa_Rosa 9.0
U_Santa_Fe 3.0
U_Paraná 2.0
U_Córdoba 4.0
U_San_Luis 8.0
U_San_Juan 6.0
U_Mendoza 7.0
U_La_Rioja 5.0
U_Viedma 14.0
U_Neuquén 10.0
U_Rawson 11.0
U_Rio_Gallegos 13.0
U_Ushuaia 12.0
U_Corrientes 1.0
Z = 9913.0

>>>
```

Thonny 3.3.6

https://thonny.org

Windows 10 (64-bit)
Python 3.7.9 (32 bit)
Tk 8.6.9

Made at the University of Tartu, Estonia, with help from the open-source community, Raspberry Pi Foundation and Cybernetica AS

Copyright (©) 2025 Aivar Annamaa
This program comes with ABSOLUTELY NO WARRANTY!
It is free software, and you are welcome to redistribute it under certain conditions, see https://opensource.org/licenses/MIT for details

OK

Programación Lineal Entera: Viajante

- *Vimos 4 alternativas para obligarlo a armar un circuito:*
  - *1 - Romper los subtours*
  - *2 - Crear los puentes*
  - *3 - Forzar una secuencia*
  - *4 - Definir un orden*

- *¿Cuál es el tamaño de cada alternativa?*

Programación Lineal Entera: Viajante

SOLUCIÓN	VARIABLES	RESTRICCIONES
Romper subtours	N. (N+1)	((2^(N+1))/2 - N - 2)
Crear puentes	N. (N+1)	((2^(N+1))/2 - N - 2)
Forzar secuencia	N. (N+1) (N)	(N+1)^2 + N+1
Definir orden	N. (N+1) + N	2 (N+1) + (N)(N-1)

N=9		N=23	
V	R	V	R
90	521	552	8388631
90	521	552	8388631
810	110	12696	600
99	92	575	554

¿Y si usamos Fuerza Bruta? ¿Cuánto más grande puede ser (N+1)! ?

Para N=9, hay 3.628.800 circuitos posibles.

Para N=23, son 620.448.401.733.239.439.360.000 = ***6,20. 10^23***

¡Es un número más grande que una CBU!

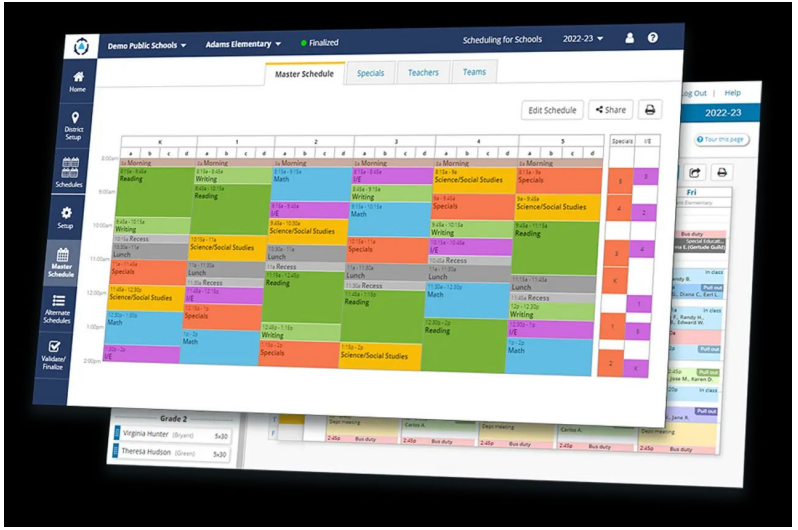
Programación Lineal Entera: Viajante

- *Tiempos de resolución*

N	ROMPER SUBTOURS	CONSTRUIR PUENTES	FORZAR UNA SECUENCIA	DEFINIR UN ORDEN (MTZ)	FUERZA BRUTA	SOFT. CONCORDE
9	7,6 s	7,3 s	0,5 s	0,6 s	31 s	< 1 s
11	15 m	20 m	1,8 s	0,6 s	1 h 45 m	< 1 s
15	?	?	2m 19 s	9 s	8 a (ESTIMADO)	< 1 s
23	?	?	5 h 15 m	12 m	2,6*10^11 a (ESTIMADO)	< 1 s

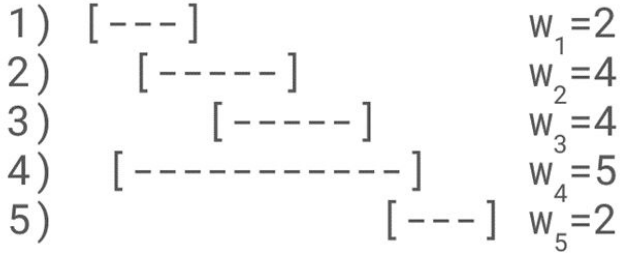
Es 15 veces la edad del universo

Resolución de Programación Lineal Entera: Weighted Interval Scheduling



Programación Lineal Entera: Scheduling

- *Programación de tareas ponderadas:*

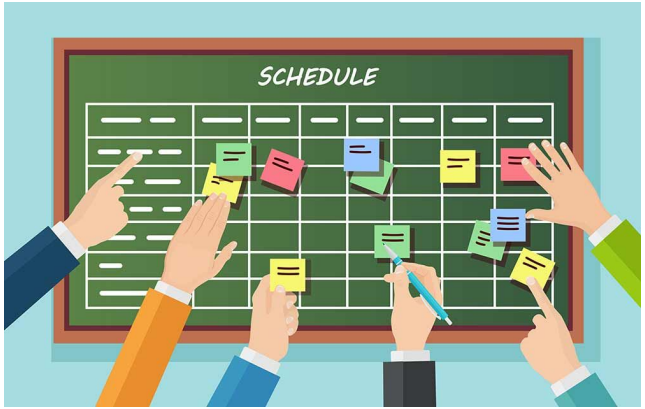


- *¿Cuáles son las variables?*
- *¿Cuál es el objetivo?*
- *¿Cuáles son las restricciones?*

Programación Lineal Entera: Scheduling

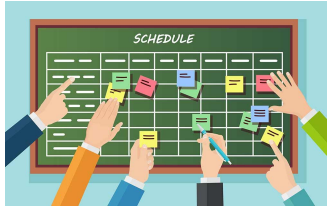
- *Programación de tareas ponderadas:*

- *Determinar qué tareas hacer.*
- *Para maximizar el beneficio.*
- *Evitando superposiciones.*



Programación Lineal Entera: Scheduling

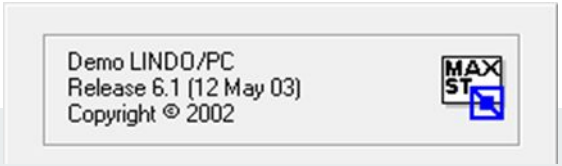
- Y_i □ *Vale 1 si elijo hacer la tarea i.*



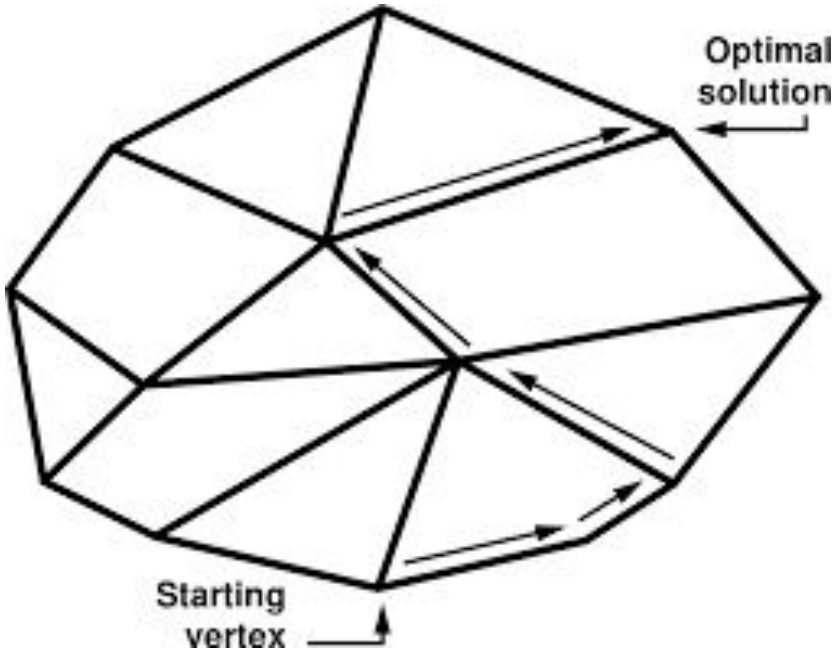
- $Z\,(Max) = \sum W_i \,.\, Y_i$
  (W_i es una constante)
- $Z\,(Max) = 2\,Y_1 + 4\,Y_2 + 4\,Y_3 + 5\,Y_4 + 2\,Y_5$

- $Y_1 + Y_2 <= 1$
- $Y_1 + Y_4 <= 1$
- $Y_2 + Y_3 <= 1$
- $Y_2 + Y_4 <= 1$
- $Y_3 + Y_4 <= 1$
- $Y_4 + Y_5 <= 1$

```
1)  [---]                  W1=2
2)    [-----]              W2=4
3)       [-----]           W3=4
4)   [-----------]         W4=5
5)                 [---]   W5=2
```



Resolución de Programación Lineal Continua: El método Simplex



Método simplex

Resolvemos el problema de Fa Caldo:

$$2X_1 + 2X_2 \leq 600$$
$$4X_2 \leq 600$$
$$2X_1 + 4X_2 \leq 800$$
$$Z_{\max} = 8X_1 + 10X_2$$



$$2X_1 + 2X_2 + S_1 = 600$$
$$4X_2 + S_2 = 600$$
$$2X_1 + 4X_2 + S_3 = 800$$
$$Z_{\max} = 8X_1 + 10X_2 + 0S_1 + 0S_2 + 0S_3$$

3 ecuaciones + 5 variables = infinitas soluciones

Pasamos de expresión algebraica a expresión matricial:

$$Z_{\max / \min} = \sum_{\forall i} c_i X_i$$
$$x_i \geq 0 \forall i$$
$$\sum_i a_{i,j} X_i = b_j, \forall j$$



$$Z_{\max / \min} = C \cdot X$$
$$X \geq 0$$
$$A \cdot X = B$$

Método simplex

En nuestro caso:

$$2X_1 + 2X_2 + S_1 = 600$$
$$4X_2 + S_2 = 600$$
$$2X_1 + 4X_2 + S_3 = 800$$
$$Z_{\max} = 8X_1 + 10X_2 + 0S_1 + 0S_2 + 0S_3$$

$$A = \begin{pmatrix} 2 & 2 & 1 & 0 & 0 \\ 0 & 4 & 0 & 1 & 0 \\ 2 & 4 & 0 & 0 & 1 \end{pmatrix} \quad X = \begin{pmatrix} X_1 \\ X_2 \\ S_1 \\ S_2 \\ S_3 \end{pmatrix} \quad B = \begin{pmatrix} 600 \\ 600 \\ 800 \end{pmatrix}$$

$$C = \begin{pmatrix} 8 & 10 & 0 & 0 & 0 \end{pmatrix}$$

Hasta acá, simplemente expresamos el planteo de diferentes maneras

Método Simplex

Armamos nuestra tabla del método simplex

$$\begin{pmatrix} C_k & X_k & B_k & A_1 & A_2 & A_3 & A_4 & A_5 \\ 0 & S_1 & 600 & 2 & 2 & 1 & 0 & 0 \\ 0 & S_2 & 600 & 0 & 4 & 0 & 1 & 0 \\ 0 & S_3 & 800 & 2 & 4 & 0 & 0 & 1 \end{pmatrix}$$

$$Z_j = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

$$Z_j - C_j = \begin{pmatrix} -8 & -10 & 0 & 0 & 0 \end{pmatrix}$$

Se obtiene de hacer $C_{fila1} * A_1 + C_{fila2} * A_1 + C_{fila3} * A_1$

Nos indica que si incorporamos a X_1 o X_2 la situación mejora, con S_1, S_2, S_3 la situación no cambia.

Método Simplex

Siguiente paso: incorporamos X_1

Puede incorporarse en su mínima cantidad según cada fila:

Fila1: 600/2 = 300; Fila2: 600/0 (no contamos); Fila3: 800/2 = 400 ➔ 300 (incorporo en fila 1)

$$\begin{pmatrix} C_k & X_k & B_k & A_1 & A_2 & A_3 & A_4 & A_5 \\ 8 & X_1 & 300 & 1 & 1 & \frac{1}{2} & 0 & 0 \\ 0 & S_2 & 600 & 0 & 4 & 0 & 1 & 0 \\ 0 & S_3 & 200 & 0 & 2 & -1 & 0 & 1 \end{pmatrix}$$

← Acá divido por el coeficiente de X_1

← Acá opero restando filas, multiplicando por constantes, para que me quede en 0

$$Z = \begin{pmatrix} 8 & 8 & 4 & 0 & 0 \end{pmatrix} \qquad Z - C = \begin{pmatrix} 0 & -2 & 4 & 0 & 0 \end{pmatrix}$$

Método simplex

Incorporamos ahora a X_2

Puede incorporarse en su mínima cantidad según cada fila:
Fila1: 300/1 = 300; Fila2: 600/4 = 150; Fila3: 200/2 = 100 ➔ 100 (incorporo en fila 3)

$$\begin{pmatrix} C_k & X_k & B_k & A_1 & A_2 & A_3 & A_4 & A_5 \\ 8 & X_1 & 200 & 1 & 0 & 1 & 0 & -\frac{1}{2} \\ 0 & S_2 & 200 & 0 & 0 & 2 & 1 & -2 \\ 10 & X_2 & 100 & 0 & 1 & -\frac{1}{2} & 0 & \frac{1}{2} \end{pmatrix}$$

Óptimo: 8 * 200 + 10 * 100 = 2600

$$Z = \begin{pmatrix} 8 & 10 & 3 & 0 & 1 \end{pmatrix}$$

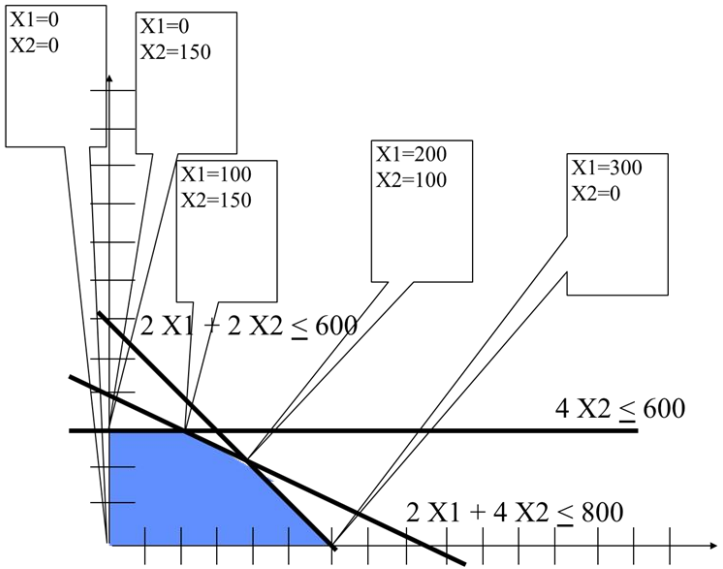
$$Z - C = \begin{pmatrix} 0 & 0 & 3 & 0 & 1 \end{pmatrix}$$

Cualquier variable que entre empeora el funcional □ **¡llegamos al óptimo!**

Método simplex

$$\begin{pmatrix} C_k & X_k & B_k & A_1 & A_2 & A_3 & A_4 & A_5 \\ 8 & X_1 & 200 & 1 & 0 & 1 & 0 & -\frac{1}{2} \\ 0 & S_2 & 200 & 0 & 0 & 2 & 1 & -2 \\ 10 & X_2 & 100 & 0 & 1 & -\frac{1}{2} & 0 & \frac{1}{2} \end{pmatrix}$$

$X_1 = 200$
$X_2 = 100$
$Z = 200 \cdot 8 + 100 \cdot 10 = 2600$



Método simplex

¿Cómo resolvemos cuando el vector nulo no es solución?

Ej: $X_1 >= 2 \rightarrow X_1 - S_1 = 2$
Esto aún no resuelve. Creamos una variable artificial:

$$X_1 - S_1 + \mu = 2$$

Para asegurarnos que la variable artificial termine valiendo 0, le ponemos un coeficiente muy grande y contrario a lo que queremos lograr:

$$Z_{\max} = \cdots - M\mu$$

Nota: si el problema no tiene solución (sistema incompatible) entonces una o más variables artificiales toman valor en la solución aparentemente “óptima”

Método simplex

Otras consideraciones

Si nuestro dominio (poliedro) es abierto y es un problema de máximo, nos va a pasar que todos los Z_i van a ser negativos (una variable “quiere entrar", pero ninguna “puede salir"), o bien cada uno de estos vale "infinito" (divido por 0). Nuevamente, no hay solución óptima porque siempre podemos seguir creciendo.

Resolución de Programación Lineal Entera: Branch and Bound



Programación Lineal Entera

- Método de planos de corte: Va eliminando los bordes del poliedro hasta llegar a una solución entera

- Método branch and bound: Para cada variable con resultado no entero, divide el problema en 2 forzándola a tomar el valor entero inferior o superior

Branch and bound

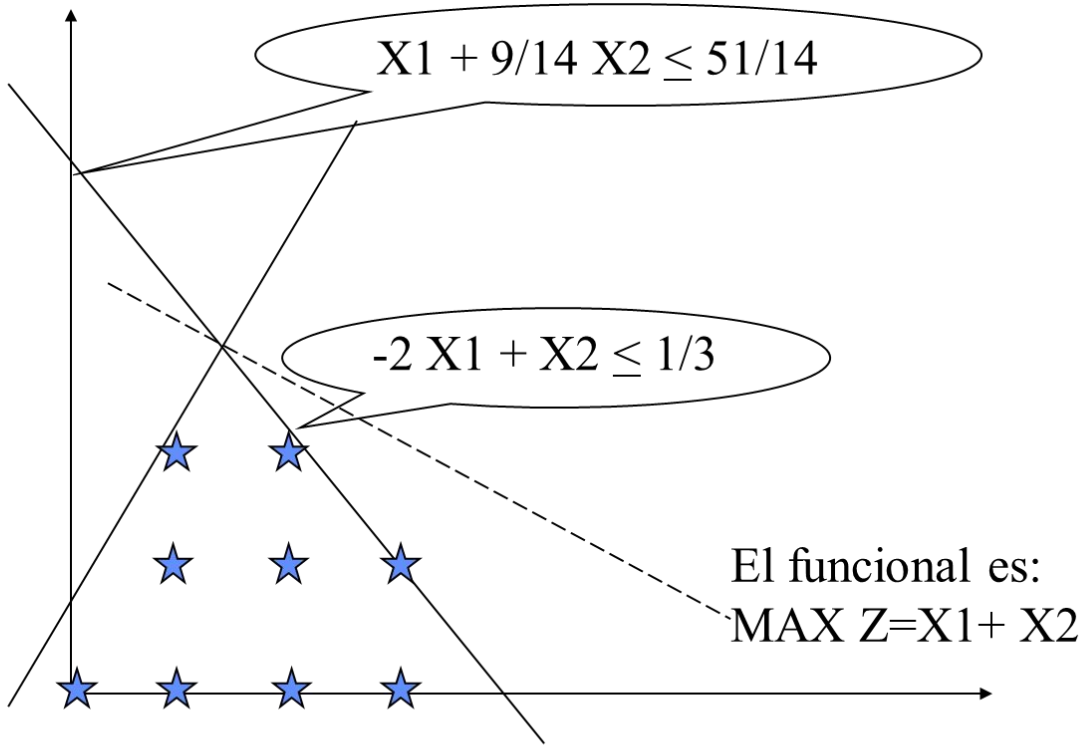
BRANCH

1. Para el nodo con mejor solución (continua) no entera, genero dos subnodos, limitando la variable no entera a los enteros más próximos.
2. Las soluciones de esos subnodos pueden ser:
   a. Incompatibles
   b. No enteras
   c. Enteras

BOUND

3. Si obtengo una solución entera, puedo “podar” del árbol las soluciones no enteras con soluciones peores.
4. Si todavía quedan nodos con soluciones no enteras sin analizar en el árbol, vuelvo al primer paso.
5. La solución entera obtenida es la óptima.

Branch and bound



Solución óptima continua: X1=3/2 X2=10/3 Z=29/6

Branch and bound

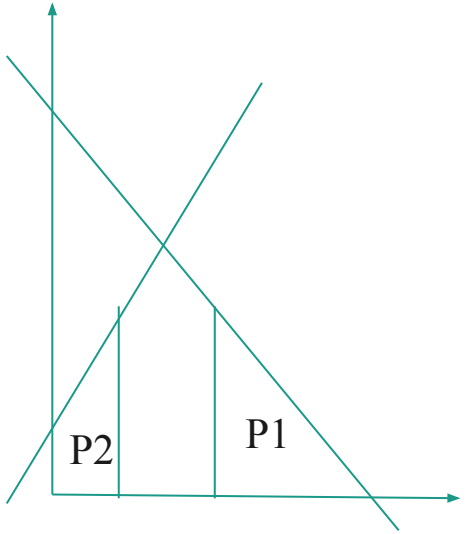
Programa P1

$Z = X_1 + X_2$ MAX

$X_1 + 9/14\, X_2 \leq 51/14$

$-2\, X_1 + X_2 \leq 1/3$

$\mathbf{X_1 \leq 1}$



Programa P2

$Z = X_1 + X_2$ MAX

$X_1 + 9/14\, X_2 \leq 51/14$

$-2\, X_1 + X_2 \leq 1/3$

$\mathbf{X_1 \geq 2}$

Óptimo:

$X_1 = 1$

$X_2 = 7/3$

$Z = 10/3$

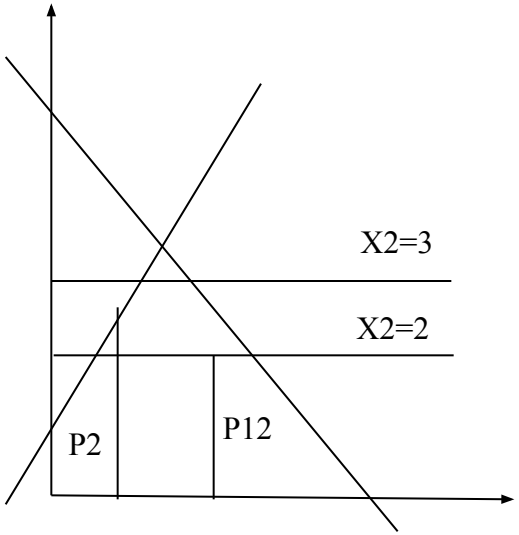
Óptimo:

$X_1 = 2$

$X_2 = 23/9$

$Z = 41/9$

Branch and bound



Programa P11

$Z = X_1 + X_2$ MAX

$X_1 + 9/14\, X_2 \leq 51/14$

$-2\, X_1 + X_2 \leq 1/3$

$X_1 \geq 2$

$\boldsymbol{X_2 \geq 3}$

INCOMPATIBLE

Programa P12

$Z = X_1 + X_2$ MAX

$X_1 + 9/14\, X_2 \leq 51/14$

$-2\, X_1 + X_2 \leq 1/3$

$X_1 \geq 2$

$\boldsymbol{X_2 \leq 2}$

Óptimo:

$X_1 = 33/14$

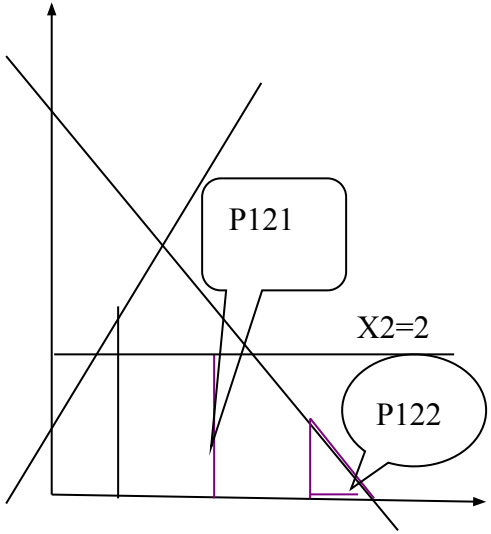
$X_2 = 2$

$Z = 61/14$

Branch and bound

Programa P121

$$Z = X_1 + X_2 \; MAX$$
$$X_1 + 9/14\, X_2 \le 51/14$$
$$-2\, X_1 + X_2 \le 1/3$$
$$X_1 \ge 2$$
$$X_2 \le 2$$
$$\boldsymbol{X_1 \le 2}$$



Programa P122

$$Z = X_1 + X_2 \; MAX$$
$$X_1 + 9/14\, X_2 \le 51/14$$
$$-2\, X_1 + X_2 \le 1/3$$
$$X_1 \ge 2$$
$$X_2 \le 2$$
$$\boldsymbol{X_1 \ge 3}$$

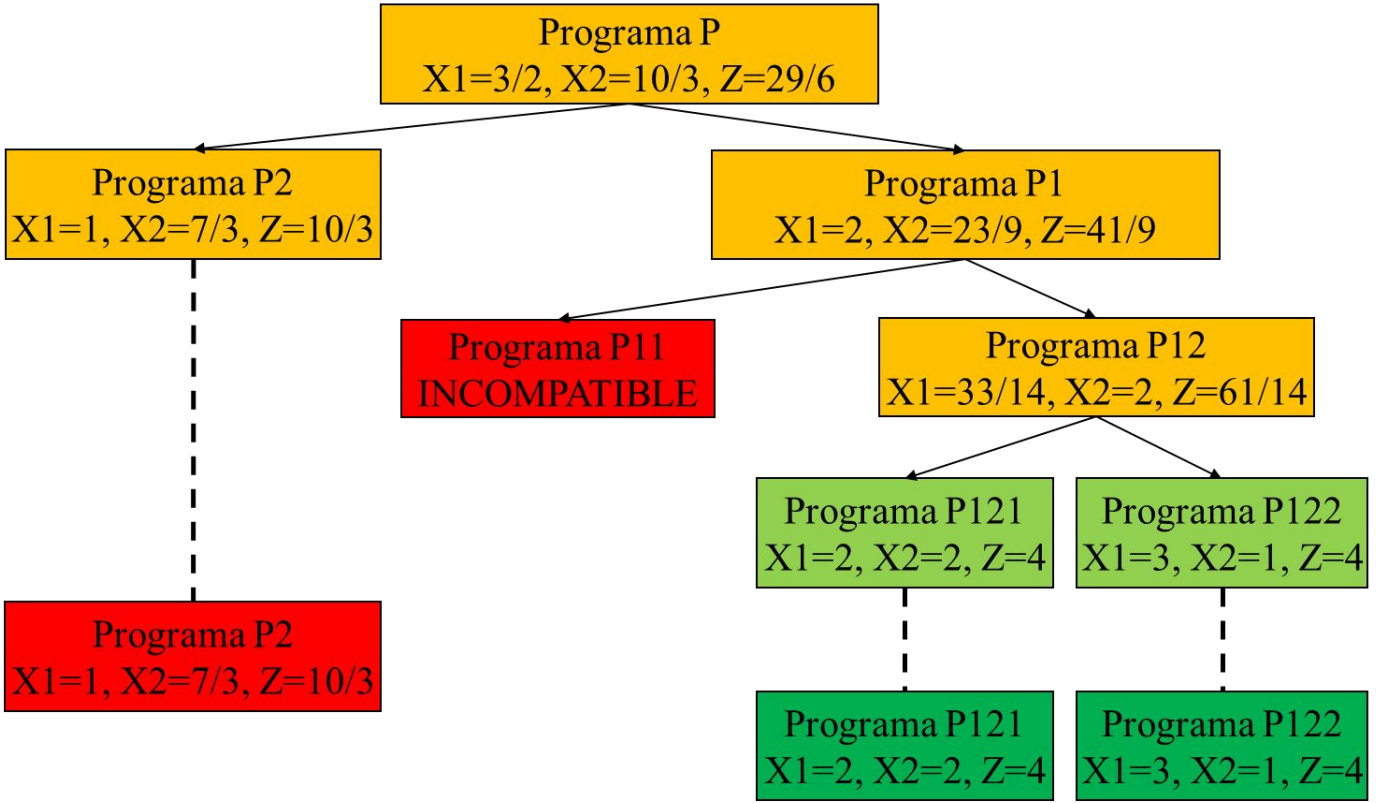
Óptimo:

$X_1 = 2$
$X_2 = 2$
$Z = 4$

Óptimo:

$X_1 = 3$
$X_2 = 1$
$Z = 4$

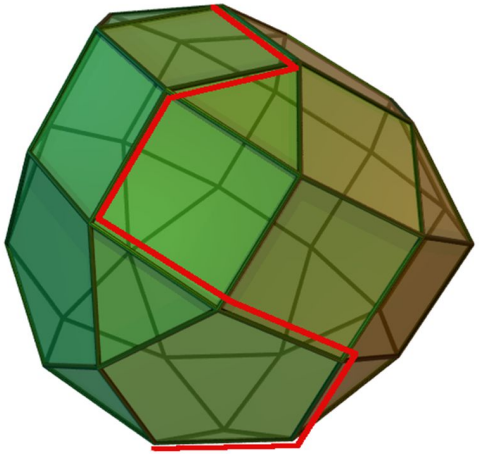
Branch and bound



Programación Lineal: Complejidad

- Programación Lineal continua:

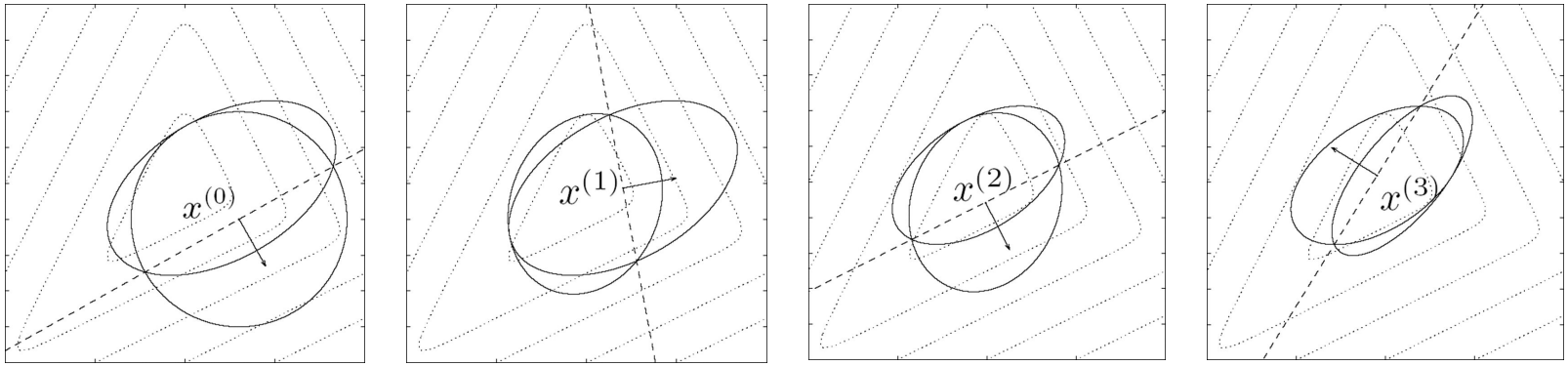
Método simplex (1947): Complejidad exponencial



Programación Lineal: Complejidad

- Programación Lineal continua:

Método del elipsoide (1979): Polinomial

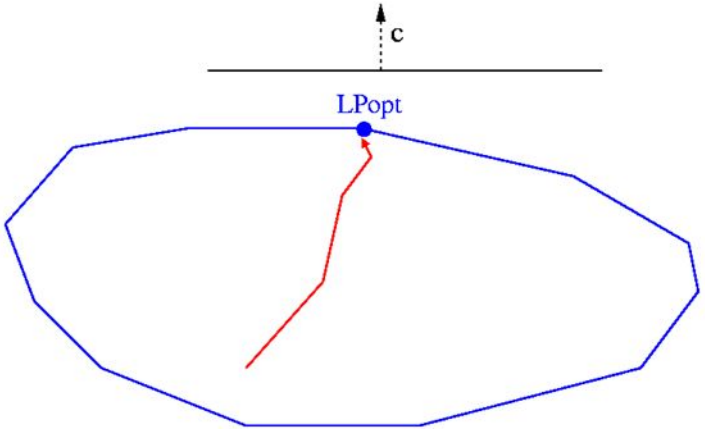


En la práctica no es útil, pero demuestra que la PL es Polinomial

Programación Lineal: Complejidad

- Programación Lineal continua:

Método del punto interior (1984): a partir del elipsoide



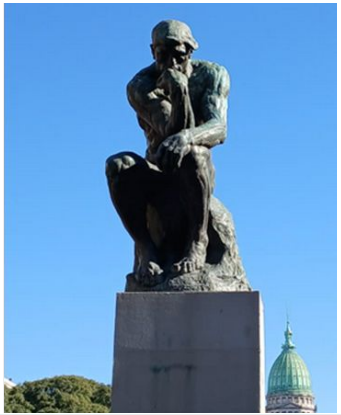
Este sí tiene uso práctico, junto con Simplex

Programación Lineal: Complejidad

- ¿Por qué usamos Simplex entonces?

  Porque FUNCIONA

  - Suele resolver problemas complejos en 3m pasos (siendo m la cantidad de restricciones)
  - Da información adicional como costos de oportunidad
  - Permite modificar el planteo de problemas ya resueltos



¿Y el nivel de complejidad?

Programación Lineal: Complejidad

- Programación Lineal Entera

  A cada variable bivalente se le asigna valor 0 y se resuelve. Luego se le asigna 1 y se resuelve otra vez.
  Si tengo una bivalente, resuelvo 2 veces el problema. Si tengo 4 variables, lo resuelvo 16 veces.

  O sea, cada problema entero equivale a 2^n problemas continuos

  O sea, es muy exponencial.



¡Muchas gracias!